

Hamiltonian Cycle Backtracking

1 Problem Definition

Hamiltonian Cycle

A **Hamiltonian cycle** in a graph is a cycle that visits every vertex exactly once and returns to the starting vertex.

For a graph with n vertices, a Hamiltonian cycle has the form

$$v_1, v_2, \dots, v_n, v_1.$$

The first n vertices are all distinct:

$$v_i \neq v_j \quad \text{whenever } i \neq j,$$

and every consecutive pair must be connected by an edge:

$$(v_i, v_{i+1}) \in E \quad \text{for } 1 \leq i < n, \quad (v_n, v_1) \in E.$$

Cycle Conditions

visit every vertex exactly once,
use only existing graph edges,
return from the last vertex to the first vertex.

2 Representation

Use an adjacency matrix

$$G[1..n][1..n],$$

where

$$G[i][j] = \begin{cases} 1, & \text{if vertices } i \text{ and } j \text{ are adjacent,} \\ 0, & \text{otherwise.} \end{cases}$$

Use a solution array

$$x[1..n],$$

where $x[k]$ is the vertex placed at position k in the cycle.

Fixing the First Vertex

Set

$$x[1] = 1.$$

This avoids generating the same cycle repeatedly by rotation.

The remaining positions are initialized as

$$x[2] = x[3] = \dots = x[n] = 0.$$

The search begins from position

$$\text{Hamiltonian}(2).$$

3 Valid Next Vertex

At position k , a candidate vertex v may be assigned to $x[k]$ only if it passes the required tests.

3.1 Adjacency to Previous Vertex

The new vertex must be adjacent to the previous vertex:

$$G[x[k-1]][v] \neq 0.$$

3.2 No Repeated Vertex

The new vertex must not already appear in the partial path:

$$v \notin \{x[1], x[2], \dots, x[k-1]\}.$$

3.3 Closing Edge at the End

If $k = n$, the last selected vertex must connect back to the first vertex:

$$G[v][x[1]] \neq 0.$$

Safety Test

A candidate v is valid at position k when

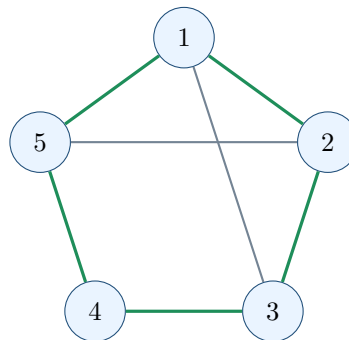
$$G[x[k-1]][v] \neq 0 \quad \text{and} \quad v \notin \{x[1], \dots, x[k-1]\}$$

and, for $k = n$, also

$$G[v][x[1]] \neq 0.$$

4 Example Graph

Consider the graph below.



One Hamiltonian cycle is

$$1, 2, 3, 4, 5, 1.$$

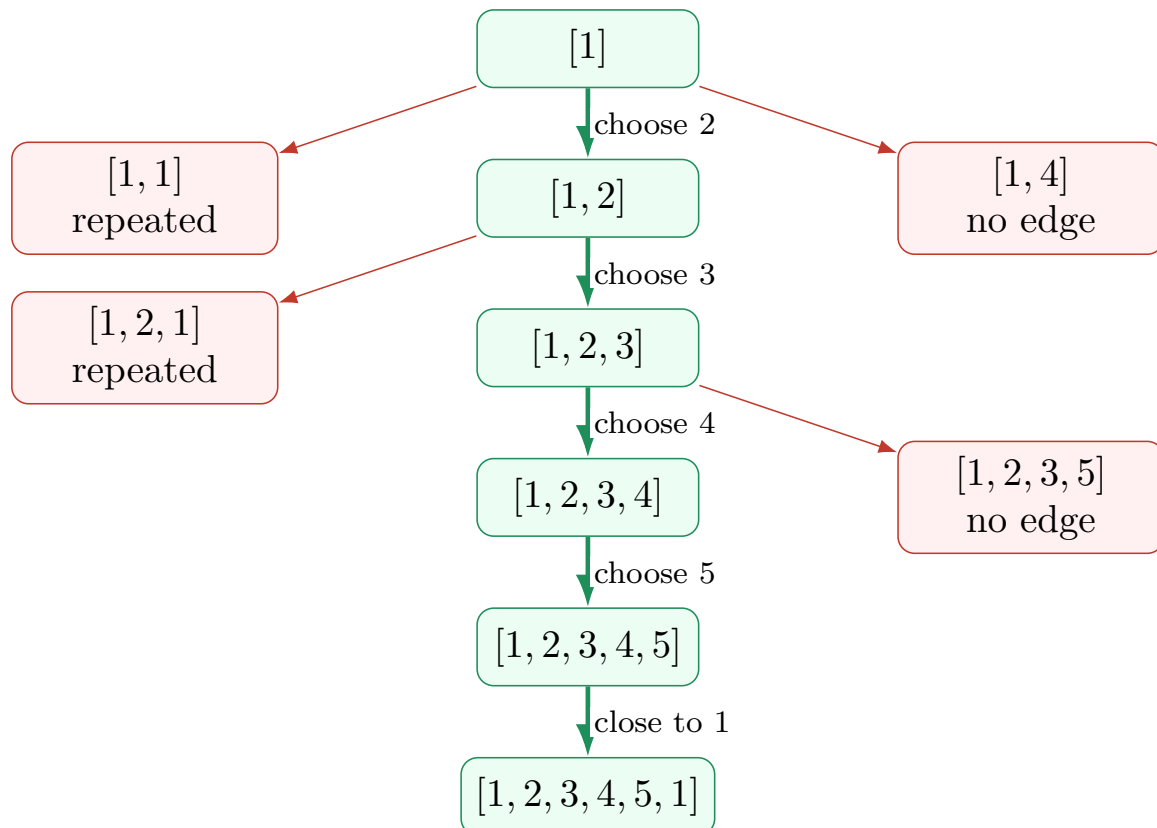
For this graph, the adjacency matrix is

$$G = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix}.$$

5 Backtracking Search

At position k , the algorithm tries candidate vertices one by one. A branch is pruned when:

- the candidate is not adjacent to the previous vertex,
- the candidate has already been used,
- the last candidate cannot connect back to $x[1]$.



Search Meaning

The green branch forms a valid Hamiltonian cycle. The red branches fail because they repeat a vertex or require a missing edge.

6 Algorithm

Main Idea of the Algorithm

The algorithm builds the cycle one position at a time in the array

$$x[1], x[2], \dots, x[n].$$

Here $x[k]$ means the vertex placed at position k of the cycle. After the array is filled, the printed cycle is

$$x[1], x[2], \dots, x[n], x[1].$$

The last $x[1]$ is not stored again in the array; it is checked using the final closing-edge test.

Why the First Vertex Is Fixed

The algorithm sets

$$x[1] = 1$$

because a cycle can be rotated. For example,

$$1, 2, 3, 4, 5, 1$$

and

$$2, 3, 4, 5, 1, 2$$

describe the same cycle with a different starting point. Fixing $x[1] = 1$ avoids generating the same cycle repeatedly by rotation.

Initialization

```
x[1] = 1
x[2..n] = 0
Hamiltonian(2)
```

Hamiltonian Search

```
Hamiltonian(k):
    while true:
        NextValue(k)

        if x[k] == 0:
            return

        if k == n:
            print x[1..n], x[1]
        else:
            Hamiltonian(k + 1)
```

Meaning of $\text{Hamiltonian}(k)$

$\text{Hamiltonian}(k)$ means: choose a valid vertex for position k . The positions $x[1], x[2], \dots, x[k-1]$ already form a valid partial path. The function tries to extend this path by assigning a suitable value to $x[k]$.

How the Recursive Search Works

After $\text{NextValue}(k)$ returns, there are two possibilities:

- If $x[k] = 0$, no valid candidate remains for position k , so the algorithm returns and backtracks.
- If $x[k] \neq 0$, a valid candidate was found. If $k = n$, the cycle is complete and is printed. Otherwise, the algorithm recursively fills position $k + 1$.

NextValue

```
NextValue(k):
    while true:
        x[k] = (x[k] + 1) mod (n + 1)

        if x[k] == 0:
            return

        if G[x[k - 1]][x[k]] != 0:
            for j = 1 to k - 1:
                if x[j] == x[k]:
                    break

            if j == k:
                if k < n:
                    return

            if k == n and G[x[n]][x[1]] != 0:
                return
```

Candidate Tests Performed by $\text{NextValue}(k)$

A candidate $x[k]$ is accepted only when it satisfies the required tests:

$$G[x[k - 1]][x[k]] \neq 0$$

checks that the new vertex is adjacent to the previous vertex,

$$x[k] \notin \{x[1], x[2], \dots, x[k - 1]\}$$

checks that the vertex has not already been used, and if $k = n$,

$$G[x[n]][x[1]] \neq 0$$

checks that the last vertex connects back to the starting vertex.

Meaning of 0

The value $x[k] = 0$ means that no more candidate vertices are available at position k . The update

$$x[k] = (x[k] + 1) \bmod (n + 1)$$

tries candidates in the order

$$1, 2, 3, \dots, n.$$

After all candidates have been tried, the value becomes 0. This tells the algorithm to backtrack to the previous position.

Backtracking Interpretation

The algorithm grows a partial path only while it remains valid. As soon as a candidate repeats a vertex, misses an edge, or fails the closing-edge test, that branch is abandoned. Then the algorithm goes back to the previous position and tries the next candidate.

7 Worked Trace

For the example graph, start with

$$x[1] = 1.$$

Choose 2:

$$G[1][2] = 1, \quad 2 \notin \{1\},$$

so

$$x[2] = 2.$$

Choose 3:

$$G[2][3] = 1, \quad 3 \notin \{1, 2\},$$

so

$$x[3] = 3.$$

Choose 4:

$$G[3][4] = 1, \quad 4 \notin \{1, 2, 3\},$$

so

$$x[4] = 4.$$

Choose 5:

$$G[4][5] = 1, \quad 5 \notin \{1, 2, 3, 4\}.$$

Since this is the last position, also check the closing edge:

$$G[5][1] = 1.$$

Answer

Therefore,

$$\boxed{1, 2, 3, 4, 5, 1}$$

is a Hamiltonian cycle.

8 Correctness Intuition

The search tree considers possible vertex permutations after fixing $x[1] = 1$. The adjacency test ensures every consecutive pair is an edge. The repetition test ensures every vertex appears at most once. The final closing-edge test ensures the path is a cycle.

1. Any printed sequence uses valid edges, has no repeated internal vertex, and returns to the start.
2. Any Hamiltonian cycle starting at vertex 1 corresponds to one branch of the search tree, so it will be found unless pruned by an invalid condition.

9 Complexity

After fixing the first vertex, the algorithm searches through permutations of the remaining vertices:

$$(n - 1)!.$$

Thus the common factorial bound is

$$\boxed{\mathcal{O}(n!)}.$$

A looser upper bound is obtained by saying that each of n positions may try up to n vertices:

$$\boxed{\mathcal{O}(n^n)}.$$

The factorial bound is sharper because the algorithm rejects repeated vertices.

The solution array and recursion depth use linear auxiliary space:

$$\boxed{\mathcal{O}(n)}.$$

10 Final Review

Hamiltonian Cycle Essentials

$x[1] = 1$ fixes the starting vertex.
 $G[x[k - 1]][x[k]] \neq 0$ checks adjacency.
 $x[k] \notin \{x[1], \dots, x[k - 1]\}$ prevents repetition.
 $G[x[n]][x[1]] \neq 0$ closes the cycle.
Time = $\mathcal{O}(n!)$, space = $\mathcal{O}(n)$.